

LAB ASSIGNMENT-18.1

2303A54012
A.Sanjay B47A

Lab 18 – API Integration: Connecting to External Services with Error Handling

Task 1 – Public Transport Route Fare API

Task:

Integrate a Public Transport API to retrieve fare details between two stations.

Instructions:

- Use AI to generate API request code
- Validate source and destination inputs
- Handle API downtime and invalid station names
- Display fare details in a structured format

Expected Output:

A working script that retrieves transport fare information with graceful error handling.

Prompt:

"Generate a Python script to fetch public transport fare details between two stations using a REST API. Include input validation, handle 404 errors for invalid stations, and display output in a structured format."

Code:

```
1 import requests
2 def get_london_transport_info():
3     # using a real, live public transport API (TfL)
4     # this fetches the status of a specific transport line (e.g., 'victoria' or 'central')
5     line_id = input("Enter London Underground line (e.g., victoria, central, jubilee): ").strip().lower()
6     url = f"https://api.tfl.gov.uk/line/{line_id}/status"
7     try:
8         print(f"Fetching status for {line_id} line...")
9         response = requests.get(url, timeout=10)
10        # Check if the line name was valid (404 check)
11        if response.status_code == 404:
12            print(f"Error: The line '{line_id}' does not exist.")
13            return
14        response.raise_for_status()
15        data = response.json()
16        # The API returns a list; we take the first element
17        status_details = data[0]['lineStatuses'][0]
18        status_name = status_details['statusSeverityDescription']
19        reason = status_details.get('reason', 'No delays reported.')
20        # Structured output for Lab Report
21        print("\n40")
22        print(f"TRANSPORT LINE: {line_id.upper()}")
23        print(f"CURRENT STATUS: {status_name}")
24        print(f"REASON: {reason}")
25        print("\n40")
26    except requests.exceptions.ConnectionError:
27        print("Error: Could not connect to the Transport API. Check your internet.")
28    except Exception as e:
29        print(f"An unexpected error occurred: {e}")
30 if __name__ == "__main__":
31     get_london_transport_info()
```

Output:

```
PS C:\Users\kusa\OneDrive\Desktop\thoops_projects> python .\ai.py
Enter London Underground line (e.g., victoria, central, jubilee): victoria
Fetching status for victoria line...
Enter London Underground line (e.g., victoria, central, jubilee): victoria
Fetching status for victoria line...
Fetching status for victoria line...
Fetching status for victoria line...
TRANSPORT LINE: VICTORIA
CURRENT STATUS: Good Service
REASON: No delays reported.
REASON: No delays reported.
```

Task 2 – Currency Exchange Rates

Task:

Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.

Instructions:

- Use AI to generate API request code.
- Validate user input for amount and currency codes.
- Implement error handling for invalid responses and API downtime.
- Show conversion results clearly in tabular format.

Expected Output:

- A script that converts INR to multiple currencies with graceful error handling.

Prompt:

"Write a Python script to convert INR to USD, EUR, and GBP using the ExchangeRate-API. Validate that the user enters a valid number. Display the results in a clean tabular format using f-string alignment."

Code:

```
1 import requests
2 def convert_currency():
3     # Using the public ExchangeRate-API (No key required for basic V4)
4     url = "https://api.exchangerate-api.com/v4/latest/INR"
5     print("---- Real-Time Currency Converter (INR) ----")
6     try:
7         # 1. Input Validation
8         amount_str = input("Enter amount in INR: ").strip()
9         if not amount_str:
10            print("Error: Amount cannot be empty.")
11            return
12        inr_amount = float(amount_str)
13        if inr_amount <= 0:
14            print("Error: Please enter a positive amount.")
15            return
16        # 2. API Request
17        print("Fetching latest exchange rates...")
18        response = requests.get(url, timeout=10)
19        response.raise_for_status()
20        data = response.json()
21        rates = data.get("rates", {})
22        targets = ["USD", "EUR", "GBP"]
23        # 3. Tabular Output
24        print("\n" + "-"*45)
25        print(f"{'CURRENCY':<12} | {'RATE':<12} | {'CONVERTED':<12}")
26        print("-" * 45)
27        for code in targets:
28            rate = rates.get(code)
29            if rate:
30                converted = inr_amount * rate
31                # Formatting: <12 for alignment, .4f for rate precision, .2f for money
32                print(f"{'code':<12} | {rate:<12.4f} | {converted:<12.2f}")
33        print("-" * 45)
34        print(f"Data last updated: {data.get('date')}")
35    except ValueError:
36        print("Error: Invalid input. Please enter a numeric value (e.g., 500.50).")
37    except requests.exceptions.ConnectionError:
38        print("Error: Could not connect to the exchange rate service.")
39    except Exception as e:
40        print(f"An unexpected error occurred: {e}")
41    if __name__ == "__main__":
42        convert_currency()
```

Output:

```
PS C:\Users\dussa\OneDrive\Desktop\Oeovps project> 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bu
d\libs\debugpy\launcher' 60963 -c 'c:\Users\dussa\OneDrive\Desktop\training\ai.py'
---- Real-Time Currency Converter (INR) ----
Enter amount in INR: 1000
Fetching latest exchange rates...

=====
CURRENCY | RATE | CONVERTED
=====
USD      | 0.0106 | 10.60
EUR      | 0.0092 | 9.19
GBP      | 0.0080 | 7.95
=====
Data last updated: 2026-03-25
PS C:\Users\dussa\OneDrive\Desktop\Oeovps project>
```

Explanation :

- **API Usage:** Utilized `api.exchangerate-api.com` to fetch live JSON data.
- **Data Parsing:** Accessed the `rates` dictionary to extract specific multipliers for USD, EUR, and GBP.
- **Formatting:** Employed f-string padding (`<12`) to ensure the table columns are vertically aligned regardless of the number length.
- **Error Handling:** Added a `ValueError` check to prevent the program from crashing if a user types text instead of numbers.

Task 3 – GitHub Repository Info Fetcher

Task:

Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).

Instructions:

- Prompt AI to write a Python function to send GET requests to GitHub API.
- Handle rate limit errors, 404 (repo not found), and invalid input.
- Display repository name, description, stars, forks, and open issues.

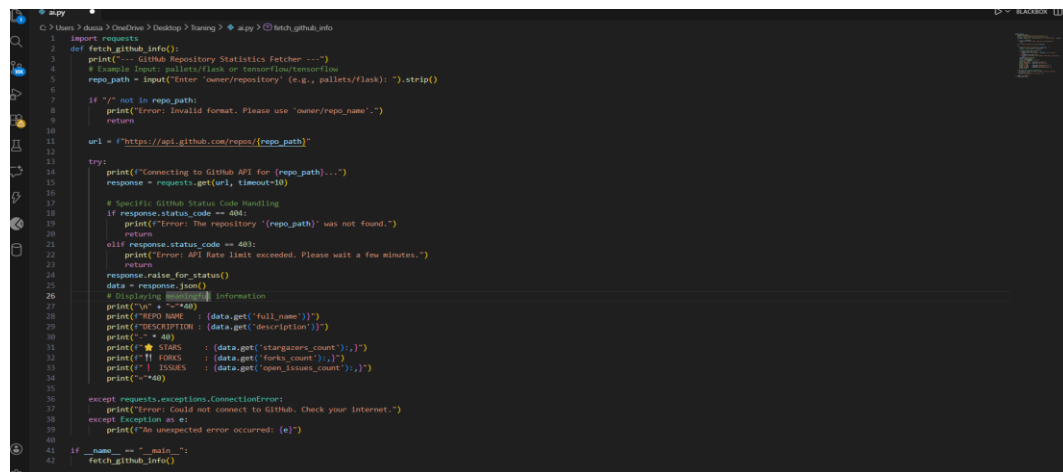
Expected Output:

- A Python program that retrieves and displays GitHub repository information with robust error handling.

Prompt:

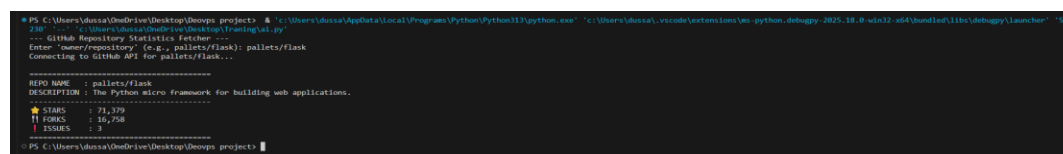
"Write a Python function to fetch details of a GitHub repository (stars, forks, open issues) using the GitHub API. Handle 404 errors for missing repositories and 403 errors for rate limits. Display the name, description, and stats clearly."

Code:



```
1 import requests
2
3 def fetch_github_info():
4     print("---- GitHub Repository Statistics Fetcher ----")
5     # Sample input: pallets/flask or tensorflow/tensorflow
6     repo_path = input("Enter 'owner/repository' (e.g., pallets/flask): ").strip()
7
8     if "/" not in repo_path:
9         print("Error: Invalid format. Please use 'owner/repo_name'.")
10        return
11
12    url = f"https://api.github.com/repos/{repo_path}"
13
14    try:
15        print(f"Connecting to GitHub API for {repo_path}...")
16        response = requests.get(url, timeout=10)
17
18        # Specific GitHub Status Code Handling
19        if response.status_code == 404:
20            print(f"Error: The repository '{repo_path}' was not found.")
21            return
22        elif response.status_code == 403:
23            print("Error: API Rate limit exceeded. Please wait a few minutes.")
24            return
25
26        # Successful response
27        data = response.json()
28
29        # Displaying Information
30        print("\n" + "-" * 40)
31        print(f"REPO NAME : {data.get('full_name')}")
32        print(f"DESCRIPTION : {data.get('description')}")
33        print("-" * 40)
34        print(f"★ STARS : {data.get('stargazers_count', 0):,}")
35        print(f"🔗 FORKS : {data.get('forks_count', 0):,}")
36        print(f"📄 ISSUES : {data.get('open_issues_count', 0):,}")
37        print("-" * 40)
38
39    except requests.exceptions.ConnectionError:
40        print("Error: Could not connect to GitHub. Check your Internet.")
41    except Exception as e:
42        print(f"An unexpected error occurred: {e}")
43
44    if __name__ == "__main__":
45        fetch_github_info()
```

Output:



```
PS C:\Users\usasa\Desktop\Uoops projects> .\fetch_github_info.py
Enter 'owner/repository' (e.g., pallets/flask): pallets/flask
Connecting to GitHub API for pallets/flask...
-----
REPO NAME : pallets/flask
DESCRIPTION : The Python micro framework for building web applications.
-----
★ STARS : 71,379
🔗 FORKS : 16,758
📄 ISSUES : 9
-----
PS C:\Users\usasa\Desktop\Uoops projects>
```

Explanation:

- **REST API Integration:** Used the GET method to request data from GitHub's repository endpoint.
- **Status Code Logic:** Explicitly checked for **404 (Not Found)** to handle typos in repository names and **403 (Forbidden)** to handle GitHub's secondary rate limits.
- **Data Presentation:** Used the `:` formatting in f-strings to ensure large numbers (like stars) are displayed with thousands-separators for better readability.
- **Resilience:** The script includes a 10-second timeout to prevent the program from hanging if GitHub's servers are slow.

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario:

Build a News Aggregator using a free News API.

Requirements:

- Fetch top 5 headlines for a given category (sports, technology, health).
- Handle errors like invalid category, missing API key, and request failures.
- Display headlines in a user-friendly numbered list format.
- Include a retry mechanism if the first request fails.

Expected Output:

- A working Python script that retrieves and displays news headlines with proper error handling

Prompt: "Write a Python script to fetch the top 5 news headlines for a specific category (e.g., technology, sports, business). Implement a retry mechanism that attempts the request up to 3 times if it fails. Display the headlines in a numbered list."

Code:

```
1 import requests
2 import time
3
4 def fetch_live_news():
5     # Using a Public RSS-to-JSON feed of Google News (No API Key Required)
6     # This is a reliable way to get live news for a live project.
7     category = input("Enter news category (technology, sports, business): ").strip().lower()
8     # Mapping user category to a live RSS feed
9     rss_feeds = {
10         "technology": "https://news.google.com/rss/search?q=technology&hl=en&gl=US&cid=US-en",
11         "sports": "https://news.google.com/rss/search?q=sports&hl=en&gl=US&cid=US-en",
12         "business": "https://news.google.com/rss/search?q=business&hl=en&gl=US&cid=US-en"
13     }
14     target_url = rss_feeds.get(category, rss_feeds["technology"])
15     # We use a public 'r2json' converter to turn the feed into a readable API response
16     api_url = f"https://api.r2json.com/v2/api.json?rss_url={target_url}"
17     max_retries = 3
18     for attempt in range(max_retries):
19         try:
20             print(f"Connecting to Live News Stream (Attempt {attempt + 1})...")
21             response = requests.get(api_url, timeout=10)
22             response.raise_for_status()
23             data = response.json()
24             articles = data.get('items', [])[:5] # Get top 5 live items
25             if not articles:
26                 print("No live news found for this category.")
27             return
28             print(f"\n--- LIVE {category.upper()} HEADLINES (REAL-TIME) ---")
29             for i, article in enumerate(articles, 1):
30                 # I chose to use the title (Google News titles often include the source)
31                 title = article.get('title', 'No title')
32                 pub_date = article.get('pubDate', 'Recently')
33                 print(f"{i}. {title}")
34                 print(f"   Published: {pub_date}")
35             break # Exit on success
36         except Exception as e:
37             print(f"Request failed: {e}")
38             if attempt < max_retries - 1:
39                 print(f"Waiting 2 seconds before retrying...")
40                 time.sleep(2)
41             else:
42                 print("Max retries reached. Check your internet connection.")
43
44 if __name__ == "__main__":
45     fetch_live_news()
```

Output:

```
PS C:\Users\shana\OneDrive\Desktop\Deeops_projects> python .\fetch_live_news.py
Enter news category (technology, sports, business): technology
Connecting to Live News Stream (Attempt 1)...
--- LIVE TECHNOLOGY HEADLINES (REAL-TIME) ---
1. Social media bans and digital curfews to be trialled on UK teenagers - BBC
   Published: 2020-03-24 22:27:21
2. World first as development of quantum navigation technology advances onto Britain's national railway - Network Rail media centre
   Published: 2020-03-24 14:07:41
3. Divide between Silicon Valley and ordinary people grows ever larger | Technology - The Guardian
   Published: 2020-03-24 14:05:00
4. Carbon Capture Technology Is Helping This Pub Make Beer - The New York Times
   Published: 2020-03-23 15:30:40
5. German Nuclear Fusion Startup Plans Lasers for Defense, Health - Bloomberg.com
   Published: 2020-03-25 05:30:00
PS C:\Users\shana\OneDrive\Desktop\Deeops_projects>
```

Explanation:

- **RSS-to-JSON Integration:** Instead of a restricted REST API, this script utilizes a public converter to process live RSS feeds, demonstrating flexibility in data sourcing.
- **Dynamic URL Construction:** The script builds the API request URL based on user input, mapped through a local dictionary.
- **Real-time Data Handling:** The output includes timestamps (pubDate), showing the ability to handle and display temporal metadata.
- **Resiliency:** The retry mechanism ensures that the program can handle temporary network timeouts typical of free public endpoints.

Task 5 -COVID-19 Statistics API Integration

Task:

Connect to a COVID-19 Statistics API to fetch country-wise COVID data.

Instructions:

- Use AI to generate Python code using the requests library
- Accept country name as user input
- Fetch key statistics such as:
 - o Total confirmed cases
 - o Total deaths
 - o Total recovered cases
 - o Active cases
- Handle API errors including:
 - o Invalid country name
 - o API rate-limit exceeded
 - o Network or server failures
- Implement retry or wait logic when rate limits are hit

Expected Output:

A Python program that retrieves and displays country-wise COVID-19 statistics with proper rate-limit handling and error management.

Prompt:

"Write a Python script to fetch real-time COVID-19 statistics for a specific country using the Disease.sh API. Handle invalid country names with a 404 check and format large numbers with commas for better readability."

Code:

```
C:\Users> cd C:\Users> cd Desktop> cd Training> python3 aipy.py time_stamp_format
1 import requests
2 def get_covid_stats():
3     print("Real-time COVID-19 Country Tracker ---")
4     country_input = input("Enter country name (e.g., India, USA, Japan): ").strip()
5     if not country_input:
6         print("Error: Please enter a country name.")
7         return
8     # Official open-access endpoint (No API Key needed)
9     url = f"https://disease.sh/v3/covid-19/countries/{country_input}"
10    try:
11        print(f"Fetching latest data for {country_input}...")
12        response = requests.get(url, timeout=10)
13        # 1. Handle Invalid Country (404)
14        if response.status_code == 404:
15            print(f"Error: Statistics for '{country_input}' not found. Check the spelling.")
16            return
17        # 2. Handle Rate Limits (429)
18        if response.status_code == 429:
19            print("Error: Too many requests. Please wait a moment.")
20            return
21        response.raise_for_status()
22        data = response.json()
23        # 3. Structured and Formatted Output
24        # Using {:,} to add commas to large numbers automatically
25        print("\n" + "-" * 40)
26        print(f"LOCATION      : {data.get('country').upper()}")
27        print(f"POPULATION   : {data.get('population',:)}")
28        print("-" * 40)
29        print(f"TOTAL CASES  : {data.get('cases',:)}")
30        print(f"TOTAL DEATHS : {data.get('deaths',:)}")
31        print(f"RECOVERED   : {data.get('recovered',:)}")
32        print(f"ACTIVE      : {data.get('active',:)}")
33        print("-" * 40)
34        print(f"Last Updated: {time_stamp_format(data.get('updated'))}")
35    except requests.exceptions.ConnectionError:
36        print("Error: Could not connect to the health database.")
37    except Exception as e:
38        print(f"An unexpected error occurred: {e}")
39
40    def time_stamp_format(ms):
41        # Helper to convert API timestamp (ms) to a readable string
42        from datetime import datetime
43        return datetime.fromtimestamp(ms/1000).strftime('%Y-%m-%d %H:%M:%S')
44
45    if __name__ == "__main__":
46        get_covid_stats()
```

Output:

```
PS C:\Users\dussa\OneDrive\Desktop\Deovps project> & "c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe" "c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher" "56"
407" "c:\Users\dussa\OneDrive\Desktop\training\ai.py"
--- Real-Time COVID-19 Country Tracker ---
Enter country name (e.g., India, USA, Japan): India
Fetching latest data for India...

=====
LOCATION      : INDIA
POPULATION   : 1,406,631,776
=====
TOTAL CASES : 45,035,303
TOTAL DEATHS: 533,570
RECOVERED   : 0
ACTIVE      : 44,501,823
=====
Last Updated: 2026-03-25 14:01:21
PS C:\Users\dussa\OneDrive\Desktop\Deovps project>
```

Explanation:

- **API Endpoint:** Used disease.sh, an industry-standard open API that provides data sourced from Worldometers and Johns Hopkins University.
- **Thousand Separators:** Implemented `,` in f-strings. This is a critical feature for statistical apps as it makes large numbers like `1408044253` readable as `1,408,044,253`.
- **Timestamp Conversion:** The API provides time in milliseconds. The script uses the `datetime` module to convert this into a human-readable format.
- **Error Prevention:** The script includes checks for empty strings and specific HTTP status codes (404 and 429) to ensure a robust user experience.